SMART HOSPITAL QUEUE MANAGEMENT SYSTEM

A Mini Project Report Using Java

Submitted in partial fulfillment of the requirements for the course

CS25C09 - Java Programming

Submitted by

MAHIMA M

Register Number: 814725104085

Department of Computer Science and Engineering

SRM TRP Engineering College (An Autonomous Institution), Tiruchirappalli

Academic Year: 2026-2027

SRM TRP Engineering College
(An AUTONOMOUS Institution)
Tiruchirappalli
Higher Order Assessment with Rubrics

Department	CSE										
Programme:	B.E	**Semester:**	03	**AY:**	2026 – 2027 [ODD]						
Faculty Handling:	Mrs.R.Sasikala,AP/CSE										
Course Code & Title:	CS25C09 – Java Programming										
Assessment Task:	Project Based Learning		**Assessment Type:**	Individual Work	**Required Hrs:**	30					
Assessment Question based on:	*Open Ended*	✓	*Problem Based Learning*		*Creative Thinking*						
	Close Ended		*Project Based Learning*	✓	*Critical Thinking*	✓					
	Any Other Type		*Challenge Based Learning*		*Level of Thinking (RBT)*	K5/ K6					
SDG to be Addressed:	SDG is Required	Yes/ No	*SDG Number (Specify or Mention Students Choice)*		*Target Number (If Applicable)*						
SDG Integration Level: (If Addressed)	*Level 1: Awareness*		*Level 2: Design Alignment*	✓	*Level 3: Impact and Measurement*						
RIASEC Context	Realistic✓	Investigative ✓	Artistic	Social	Enterprising	Conventional					
Presentation Format:	Demonstration/Report/PowerPoint										
Max Marks:	**50**										
Relevant Course Outcome(s):	CO1, CO2, CO3, CO4, CO5										
Submission Deadline:	05.11.2026										
Submitted by (Name & Register No.):	Name: MAHIMA M Register No: 814725104085										
Project Title:											

Rubrics for Evaluation:

Assessment Criteria	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Framing (10)	9–10: Clearly defines the problem and objectives.	7–8: Defines the problem with minor gaps.	5–6: Shows basic understanding.	0–4: Problem is unclear.
Conceptual Foundation & Background Knowledge (10)	9–10: Demonstrates strong conceptual knowledge and relevant background.	7–8: Shows good understanding with minor gaps.	5–6: Demonstrates basic knowledge.	0–4: Shows insufficient knowledge.
Analytical Reasoning & Synthesis (15)	13–15: Analyses information logically and develops well-supported conclusions.	10–12: Provides reasonable analysis with minor gaps.	7–9: Provides basic analysis with limited justification.	0–6: Analysis is weak or incomplete.
Documentation & Clarity of Explanation (10)	9–10: Presents clear, structured ,complete documentation and upload the projects in Github Repository.	7–8: Documentation is clear with minor omissions.	5–6: Documentation is understandable but lacks detail.	0–4: Documentation is unclear or incomplete.
Reflective Learning Insight (5)	5: Clearly explains learning, challenges and improvements.	4: Explains learning with some reflection.	3: Identifies basic learning outcomes.	0–2: Provides little or no reflection.

Individual Mark Allocation:

Register No: 814725104085		**Name:** MAHIMA M			
Assessment Criteria	**Excellent**	**Good**	**Satisfactory**	**Needs Improvement**	**Total**
Problem Understanding & Framing (10)					
Conceptual Foundation & Background Knowledge (10)					
Analytical Reasoning & Synthesis (15)					
Documentation & Clarity of Explanation (10)					
Reflective Learning Insight (5)					
				Grand Total	

Signature of the Faculty

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Existing and Proposed Systems
4. Hardware and Software Requirements
5. System Design
6. Java Concepts Used
7. Implementation
8. Sample Java Program
9. Sample Output
10. Testing
11. Analysis and Discussion
12. SDG Alignment
13. Advantages and Limitations
14. Conclusion
15. Future Enhancements
16. Reflective Learning Insight
17. GitHub Repository
18. References
19. Certificate

Abstract

Long waiting times and unmanaged queues are among the most common problems in hospitals and clinics. Patients usually collect a paper slip or stand in a physical line, and staff have no easy way to tell how many people are waiting, who should be seen first, or how long a new arrival will have to wait. Medical emergencies are sometimes delayed simply because they joined the line late.

The Smart Hospital Queue Management System is a Java-based console application that digitises this process. Each patient is registered once, receives a unique token number, and is placed in one of two queues: an emergency queue or a normal queue. When the doctor is ready, the system always calls emergency patients first and then the remaining patients in the order of arrival. The application also lets staff view the waiting list, find a patient's position from the token number, cancel a token, and see simple statistics including an estimated waiting time.

The project applies object-oriented programming concepts such as classes, objects, constructors, encapsulation and methods, together with the Java Collections Framework (Queue, LinkedList and ArrayList), loops, conditional statements and the Scanner class. The first version stores data in memory and uses a console interface, which keeps it suitable for beginners; it can later be extended with Java Swing, JDBC and SMS notifications. The project is aligned with UN Sustainable Development Goal 3 (Good Health and Well-being).

Chapter 1 – Introduction

A hospital outpatient department (OPD) serves a large number of patients every day. Each patient needs registration, consultation with a doctor and, in many cases, laboratory tests or pharmacy services. When the number of arrivals is higher than the number of doctors available, queues form. If these queues are not managed properly, patients wait for long hours, crowd the corridors, and become frustrated, while genuinely urgent cases may not get attention in time.

In many small and medium hospitals, the queue is still managed with handwritten registers, paper tokens or by simply asking patients to stand in a line. This depends fully on the receptionist and offers no visibility to patients or to the management.

The Smart Hospital Queue Management System uses Java to automate the basic queue operations. It issues tokens automatically, separates emergency cases from normal cases, calls patients in the correct order, and provides a quick summary of the current queue.

Problem Statement

Manual queue handling in hospitals leads to long and uncertain waiting times, duplicate or lost tokens, no priority for critical patients, and no reliable way to estimate when a patient will be called. Staff also spend time answering repeated questions such as "how many people are before me?"

Therefore, a simple Java application is proposed that registers patients, assigns tokens, gives priority to emergency cases, and lets staff manage the queue efficiently.

Objectives

- To develop a console-based hospital queue management application using Java.
- To register patients and generate unique, sequential token numbers.
- To maintain separate queues for emergency and normal patients.
- To call the next patient with emergency cases always served first.
- To display the waiting list and the position of any token.
- To cancel a token when a patient leaves the queue.
- To show queue statistics and an estimated waiting time.
- To apply object-oriented programming concepts and the Collections Framework in a practical application.

Chapter 2 – Existing and Proposed Systems

Existing System

In the existing manual system:

- Patients stand in a physical line or collect a paper slip at the reception.
- Emergency patients are identified only by the receptionist's judgement and shouting or announcement.
- Patients have no way of knowing their position or expected waiting time.
- Lost or duplicated slips cause disputes and confusion.
- There is no record of how many patients were served in a day.

Proposed System

The proposed Java application provides:

- Automatic token generation for every registered patient.
- Two separate queues so that emergency cases are always called first.
- A menu-driven interface that is easy for reception staff to use.
- Token search to show the exact position of a patient in the waiting list.
- Token cancellation and basic statistics, including estimated waiting time.
- Scope for future enhancement using graphical interfaces, databases and SMS alerts.

Chapter 3 – Hardware and Software Requirements

Requirement	Specification
Hardware	Computer or laptop
Processor	Standard dual-core processor or better
RAM	4 GB or above recommended
Operating system	Windows or Linux
Programming language	Java
JDK	JDK 8 or later
IDE	Eclipse, IntelliJ IDEA, NetBeans, or VS Code
Data storage	In-memory collections in the sample version
Version control	Git and GitHub repository

Chapter 4 – System Design

System Architecture

The main controller reads the user's choice from the menu and calls the corresponding module. The modules work on two queues (emergency and normal) and on a list that records the patients already served.

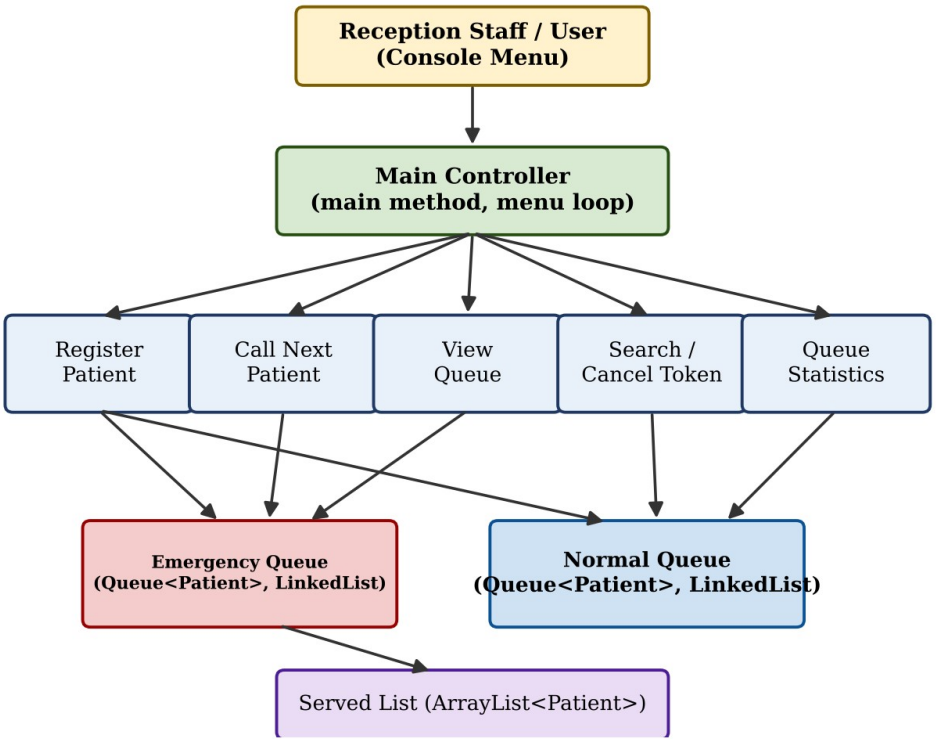


Fig. 4.1 System Architecture

Program Flow

The flow chart below shows how a patient is registered, placed in the correct queue, and later called by the system.

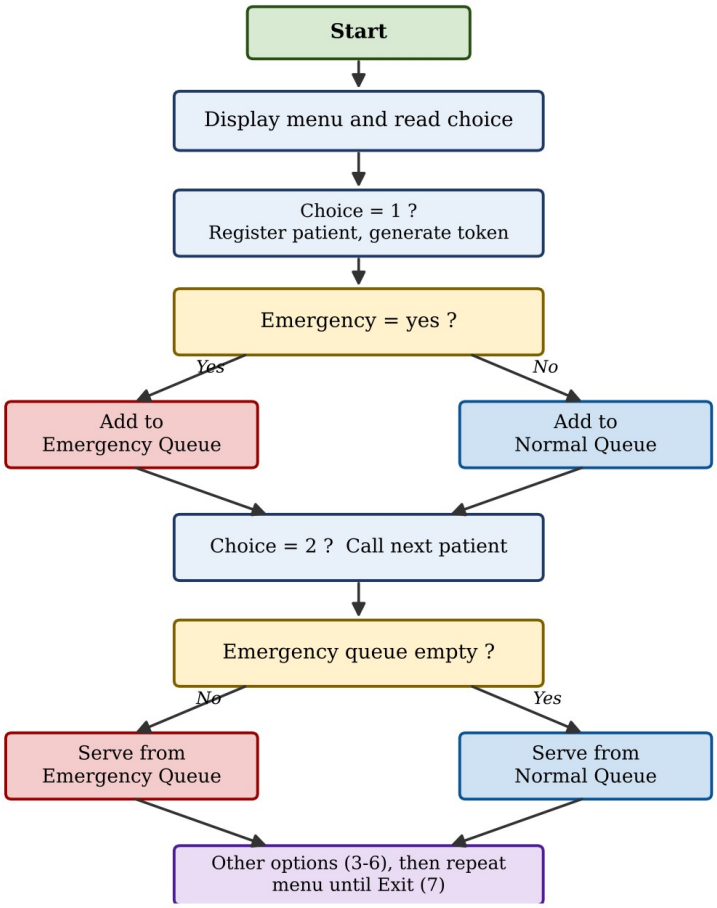


Fig. 4.2 Program Flow of the Queue Management System

Modules

Patient Module

The Patient class stores the token number, name, age, department and emergency status of a patient.

Registration Module

Accepts patient details, generates the next token number and adds the patient to the emergency or normal queue.

Call Next Module

Removes the patient at the front of the emergency queue, or of the normal queue if no emergency is waiting, and records the patient as served.

View Queue Module

Displays all waiting patients, with emergency patients listed first.

Search Token Module

Finds a token in the waiting list and reports the patient's position.

Cancel Token Module

Removes a token from whichever queue contains it.

Statistics Module

Shows the number of patients waiting and served, and an estimated waiting time based on 10 minutes per patient.

Chapter 5 – Java Concepts Used

Java concept	Application in the project
Class	Patient and SmartHospitalQueue
Object	Individual patient records created at registration
Encapsulation	Patient information stored together as fields of one class
Constructor	Patient(...) initialises token, name, age, department and emergency status
Methods	Register, call next, view, search, cancel and statistics operations
Queue and LinkedList	Two FIFO queues hold the waiting emergency and normal patients
ArrayList	Stores the list of patients already served
Enhanced for loop	Traverses the queues to display and search patients
Lambda expression	removeIf(p -> p.token == token) cancels a token
Conditional statements	Decide the queue, the next patient and the result of a search
switch and do-while	Menu selection and repeating the menu until Exit
Static members	Shared queues, token counter and Scanner object
Scanner	Reads input from the user

Chapter 6 – Implementation

The application follows these steps:

1. Start the Java program.
2. Display the main menu.
3. Read the user's choice.
4. Execute the selected operation on the queues.
5. Display the result.
6. Return to the main menu until the user selects Exit.

Algorithm

Step 1: Start

Step 2: Create two empty queues (emergency and normal), an empty served list, and set the token counter to 1.

Step 3: Display the menu.

Step 4: Read the user's choice.

Step 5: If choice is 1, read patient details, generate a token, and add the patient to the emergency queue if the case is an emergency, otherwise to the normal queue.

Step 6: If choice is 2, remove the first patient of the emergency queue; if it is empty, remove the first patient of the normal queue; add the patient to the served list.

Step 7: If choice is 3, display the emergency queue followed by the normal queue.

Step 8: If choice is 4, search the queues for the token and display its position.

Step 9: If choice is 5, remove the token from the queue that contains it.

Step 10: If choice is 6, display the waiting count, served count and estimated waiting time.

Step 11: If choice is 7, exit the program.

Step 12: Repeat the menu until Exit is selected.

Step 13: Stop.

Chapter 7 – Sample Java Program

The following is a complete console application. Save it in a file named SmartHospitalQueue.java, compile it with javac SmartHospitalQueue.java and run it with java SmartHospitalQueue.

```
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

class Patient {
    int token;
    String name;
    int age;
    String department;
    boolean emergency;

    Patient(int token, String name, int age, String department, boolean emergency) {
        this.token = token;
        this.name = name;
        this.age = age;
        this.department = department;
        this.emergency = emergency;
    }

    void display() {
        System.out.println(token + " | " + name + " | " + age + " | "
                + department + " | " + (emergency ? "Emergency" : "Normal"));
    }
}

public class SmartHospitalQueue {
    static Queue<Patient> emergencyQueue = new LinkedList<>();
    static Queue<Patient> normalQueue = new LinkedList<>();
    static ArrayList<Patient> servedList = new ArrayList<>();
    static Scanner sc = new Scanner(System.in);
    static int nextToken = 1;

    static void registerPatient() {
        sc.nextLine();
        System.out.print("Enter Patient Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Age: ");
        int age = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Department: ");
        String dept = sc.nextLine();
        System.out.print("Is this an emergency? (y/n): ");
        boolean emergency = sc.nextLine().trim().equalsIgnoreCase("y");

        Patient p = new Patient(nextToken++, name, age, dept, emergency);
        if (emergency) {
            emergencyQueue.add(p);
        } else {
            normalQueue.add(p);
        }
        System.out.println("Registered. Token number: " + p.token);
    }

    static void callNextPatient() {
```

```
    Patient p = null;
    if (!emergencyQueue.isEmpty()) {
        p = emergencyQueue.poll();
    } else if (!normalQueue.isEmpty()) {
        p = normalQueue.poll();
    }
    if (p == null) {
        System.out.println("No patients waiting.");
        return;
    }
    servedList.add(p);
    System.out.print("Now serving -> ");
    p.display();
}

static void viewQueue() {
    if (emergencyQueue.isEmpty() && normalQueue.isEmpty()) {
        System.out.println("Queue is empty.");
        return;
    }
    System.out.println("Token | Name | Age | Department | Type");
    for (Patient p : emergencyQueue) {
        p.display();
    }
    for (Patient p : normalQueue) {
        p.display();
    }
}

static void searchToken() {
    System.out.print("Enter Token Number: ");
    int token = sc.nextInt();
    int position = 1;
    for (Patient p : emergencyQueue) {
        if (p.token == token) {
            System.out.println("Position in queue: " + position);
            return;
        }
        position++;
    }
    for (Patient p : normalQueue) {
        if (p.token == token) {
            System.out.println("Position in queue: " + position);
            return;
        }
        position++;
    }
    System.out.println("Token not found in waiting list.");
}

static void cancelToken() {
    System.out.print("Enter Token Number to cancel: ");
    int token = sc.nextInt();
    boolean removed = emergencyQueue.removeIf(p -> p.token == token)
            || normalQueue.removeIf(p -> p.token == token);
    System.out.println(removed ? "Token cancelled." : "Token not found.");
}

static void showStatistics() {
    int waiting = emergencyQueue.size() + normalQueue.size();
    System.out.println("Patients waiting : " + waiting);
```

```
        System.out.println("Patients served  : " + servedList.size());
        System.out.println("Estimated wait for a new normal patient: "
                + (waiting * 10) + " minutes");
    }

    public static void main(String[] args) {
        int choice;
        do {
            System.out.println("\n--- SMART HOSPITAL QUEUE MANAGEMENT ---");
            System.out.println("1. Register Patient");
            System.out.println("2. Call Next Patient");
            System.out.println("3. View Waiting Queue");
            System.out.println("4. Search Token Position");
            System.out.println("5. Cancel Token");
            System.out.println("6. Queue Statistics");
            System.out.println("7. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    registerPatient();
                    break;
                case 2:
                    callNextPatient();
                    break;
                case 3:
                    viewQueue();
                    break;
                case 4:
                    searchToken();
                    break;
                case 5:
                    cancelToken();
                    break;
                case 6:
                    showStatistics();
                    break;
                case 7:
                    System.out.println("Thank you!");
                    break;
                default:
                    System.out.println("Invalid choice.");
            }
        } while (choice != 7);
        sc.close();
    }
}
```

Explanation of the Program

- Patient is a class that represents a patient; its constructor initialises the token, name, age, department and emergency status.
- emergencyQueue and normalQueue are Queue<Patient> objects implemented with LinkedList, so patients leave in the same order in which they joined (FIFO).
- registerPatient() reads patient details, creates a Patient object with the next token and adds it to the correct queue. The first nextLine() clears the newline left by the menu choice.

- callNextPatient() uses poll() to remove the patient from the front of the emergency queue, or the normal queue when no emergency is waiting.
- viewQueue() and searchToken() traverse both queues with an enhanced for loop; the position counter continues from the emergency queue into the normal queue.
- cancelToken() uses removeIf() with a lambda expression to remove the matching token.
- showStatistics() reports the waiting count, served count and an estimated waiting time of 10 minutes per waiting patient.
- The switch statement selects the operation and the do-while loop repeats the menu until the user exits.

Chapter 8 – Sample Output

Illustrative console output from the sample program (replace with your own screenshots after running the program):

```
--- SMART HOSPITAL QUEUE MANAGEMENT ---
1. Register Patient
2. Call Next Patient
3. View Waiting Queue
4. Search Token Position
5. Cancel Token
6. Queue Statistics
7. Exit
Enter your choice: 1
Enter Patient Name: Ravi Kumar
Enter Age: 45
Enter Department: Cardiology
Is this an emergency? (y/n): n
Registered. Token number: 1
--- SMART HOSPITAL QUEUE MANAGEMENT ---
Enter your choice: 1
Enter Patient Name: Meena S
Enter Age: 30
Enter Department: Orthopedics
Is this an emergency? (y/n): n
Registered. Token number: 2
--- SMART HOSPITAL QUEUE MANAGEMENT ---
Enter your choice: 1
Enter Patient Name: Arun P
Enter Age: 60
Enter Department: Emergency Ward
Is this an emergency? (y/n): y
Registered. Token number: 3
--- SMART HOSPITAL QUEUE MANAGEMENT ---
Enter your choice: 3
Token | Name | Age | Department | Type
3 | Arun P | 60 | Emergency Ward | Emergency
1 | Ravi Kumar | 45 | Cardiology | Normal
2 | Meena S | 30 | Orthopedics | Normal
--- SMART HOSPITAL QUEUE MANAGEMENT ---
Enter your choice: 4
Enter Token Number: 2
Position in queue: 3
--- SMART HOSPITAL QUEUE MANAGEMENT ---
Enter your choice: 2
Now serving -> 3 | Arun P | 60 | Emergency Ward | Emergency
--- SMART HOSPITAL QUEUE MANAGEMENT ---
Enter your choice: 2
Now serving -> 1 | Ravi Kumar | 45 | Cardiology | Normal
--- SMART HOSPITAL QUEUE MANAGEMENT ---
Enter your choice: 6
Patients waiting : 1
Patients served  : 2
Estimated wait for a new normal patient: 10 minutes
```

Observe that token 3 (emergency) is served before tokens 1 and 2, although it was registered last.

Chapter 9 – Testing

Test case	Input / Action	Expected result
Register normal patient	Enter details, emergency = n	Token generated; added to normal queue
Register emergency patient	Enter details, emergency = y	Token generated; added to emergency queue
Token sequence	Register three patients	Tokens 1, 2 and 3 issued in order
View queue	Select option 3	Emergency patients listed before normal patients
Call next with emergency waiting	Select option 2	Emergency patient is served first
Call next, only normal waiting	Select option 2	Oldest normal patient is served
Call next, empty queues	Select option 2	"No patients waiting." message
Search existing token	Enter a valid token	Correct position displayed
Search invalid token	Enter unknown token	"Token not found in waiting list."
Cancel token	Enter a valid token	Token removed from queue
Cancel invalid token	Enter unknown token	"Token not found." message
Statistics	Select option 6	Waiting count, served count and estimate shown
Invalid menu choice	Enter 9	"Invalid choice." message
Exit	Select option 7	Program terminates

These are expected test cases; the student should run the program and record the actual results.

Chapter 10 – Analysis and Discussion

Choice of Data Structure

A queue follows the first-in-first-out principle, which is exactly how a fair waiting line works. Java's Queue interface provides add() to join the line and poll() to remove the patient at the front, both in constant time when implemented with LinkedList. An ArrayList was avoided for the waiting line because removing from the front of an ArrayList requires shifting all remaining elements, which takes linear time.

Two Queues versus One Priority Queue

Two designs were considered for emergency handling. A single PriorityQueue ordered by priority and arrival time would also work, but it needs a comparator and a tie-breaking rule to keep arrival order among equal-priority patients. Using two simple queues keeps the logic easy to read and test: emergency cases are always checked first, and within each queue the order of arrival is preserved automatically. For a system with only two priority levels, two queues give the same result with less code. If more levels such as triage categories are added in the future, a PriorityQueue becomes the better choice.

Time Complexity

Operation	Complexity	Reason
Register patient	O(1)	add() at the end of a LinkedList
Call next patient	O(1)	poll() removes the head
View queue	O(n)	Every waiting patient is displayed
Search token	O(n)	Linear scan of both queues
Cancel token	O(n)	removeIf() scans the queue

Observations and Limitations of the Priority Rule

Strict priority gives emergency patients immediate attention, but if emergencies arrive continuously, normal patients may wait for a very long time (starvation). In real hospitals, this is controlled by capacity planning and by limiting what counts as an emergency. The estimated waiting time of 10 minutes per patient is a fixed assumption; a real system would use the average consultation time recorded in the served list.

Chapter 11 – SDG Alignment

The project is aligned with UN Sustainable Development Goal 3 – Good Health and Well-being, in particular Target 3.8, which aims at access to quality essential health-care services. Reducing waiting time and giving priority to urgent cases improves access to care at the point of service. The integration level is Level 2 (Design Alignment): the SDG influences the design decision of separating emergency and normal queues.

Design feature	**Contribution to SDG 3**
Emergency queue served first	Critical patients receive timely attention
Token and position search	Less crowding and uncertainty in waiting areas
Statistics and waiting estimate	Helps management plan staff and consultation hours
Simple console interface	Can be used by small clinics with basic computers

For Level 3 (Impact and Measurement), future versions can measure the average waiting time before and after using the system.

Chapter 12 – Advantages and Limitations

Advantages

- Easy to understand and use.
- Gives priority to emergency patients.
- Removes dependence on paper slips and manual announcements.
- Patients can know their position in the queue.
- Demonstrates Java OOP concepts and the Collections Framework.
- Can be extended with a graphical interface and database.

Limitations

- Patient records are lost when the program terminates.
- Only one counter or doctor is supported.
- No login system for reception staff.
- The waiting time is a fixed estimate of 10 minutes per patient.
- Only two priority levels are supported, and invalid input (for example, text instead of a number) is not handled.

Chapter 13 – Conclusion

The Smart Hospital Queue Management System demonstrates how Java programming can be used to solve a real-world problem. The project implements token generation, priority-based calling, queue display, token search, cancellation and statistics using classes, objects, constructors, methods, collections, loops and conditional statements.

Through this project, the student gains practical experience in selecting a suitable data structure, designing a menu-driven application, organising code into methods, and testing program functionality. The project also shows how a simple software solution can contribute to better healthcare service and serves as a foundation for developing larger Java applications.

Chapter 14 – Future Enhancements

The project can be improved by adding:

- Java Swing forms for a graphical user interface and a display board showing the current token.
- JDBC connectivity with MySQL for permanent storage of patient and visit records.
- Separate queues for each doctor or department.
- Priority levels based on triage categories using PriorityQueue.
- Estimated waiting time calculated from actual consultation durations.
- SMS or email alerts when a patient's turn is near.
- Online appointment booking and mobile app integration.
- Reports on daily patients, average waiting time and peak hours.
- Input validation and exception handling.

Chapter 15 – Reflective Learning Insight

What I Learned

I learned how to model a real-world process with Java classes and collections, how the FIFO behaviour of a Queue matches a waiting line, and how a small design decision, such as using two queues, can satisfy an important requirement. I also practised breaking a program into methods, using a menu loop, and preparing test cases.

Challenges Faced

- Handling the leftover newline after reading a number with Scanner, which caused the name input to be skipped.
- Deciding between a PriorityQueue and two queues for emergency handling.
- Calculating a patient's position across two separate queues.

Improvements Planned

In the next version, I plan to add file or database storage, input validation, a Swing interface and per-doctor queues, and to measure the actual reduction in waiting time.

GitHub Repository

As required by the assessment rubrics, the complete source code and this report are uploaded to a GitHub repository.

Repository link: https://github.com/Mahima-1212/SmartHospitalQueue_Updated_Demo.git

Suggested repository contents:

- SmartHospitalQueue.java – complete source code
- README.md – project description, steps to compile and run, and sample output
- Report (PDF/DOCX) and output screenshots

References

- Herbert Schildt, Java: The Complete Reference.
- Oracle Java Documentation: https://docs.oracle.com/en/java/
- Java Collections Framework: https://docs.oracle.com/javase/tutorial/collections/
- Java Queue Interface: https://docs.oracle.com/javase/tutorial/collections/interfaces/queue.html
- Java Swing Tutorial: https://docs.oracle.com/javase/tutorial/uiswing/
- United Nations, Sustainable Development Goal 3: https://sdgs.un.org/goals/goal3